

MASTÈRE DÉVELOPPEMENT & DEVOPS

Rapport de veille technologique Frontend

Étude, expérimentation et choix
d'une technologie frontend

Étudiant : Fallou Gaye

Formation : Mastère Développement & DevOps

Établissement : La Plateforme

Année : 2026–2027

React • Vue.js • Angular

Choix retenu : React avec Next.js

Table des matières

Introduction	2
1 Contexte et critères de décision	2
2 Comparaison initiale des technologies	3
2.1 React	3
2.2 Vue.js	4
2.3 Angular	5
3 Synthèse comparative	5
4 Notre choix : React	6
5 React en profondeur	7
5.1 La philosophie des composants	7
5.2 État et interactions	7
5.3 Flux de données	8
6 Pourquoi Next.js autour de React ?	8
7 Axios : une dépendance volontaire	9
8 Architecture expérimentée	10
9 Retour d'expérience	12
9.1 Courbe d'apprentissage	12
9.2 Séparation des responsabilités	12
9.3 Communication avec le backend	12
10 Organisation et priorisation des User Stories	13
11 Organisation du projet dans le temps	13
12 Maintenance, écosystème et pérennité	14
13 Collaboration et arrivée d'un nouveau développeur	15
14 Limites de notre étude	15
15 Recommandation finale	16
Conclusion	16
Sources	18

Introduction

Dans le cadre de cette veille technologique, nous nous plaçons dans la situation d'une équipe technique chargée de lancer un nouveau produit au sein d'une start-up.

L'un des premiers choix importants concerne la technologie frontend. Trois solutions ont été étudiées : **React**, **Vue.js** et **Angular**.

L'objectif de cette veille n'est pas simplement de choisir la technologie la plus populaire. Le choix doit être cohérent avec les besoins du produit, les compétences de l'équipe, le temps de développement, la maintenance et l'évolution future de l'application.

Notre démarche s'est donc organisée en trois étapes.

Nous avons d'abord étudié les trois technologies afin d'identifier leurs principales caractéristiques, leurs avantages et leurs limites.

Nous avons ensuite retenu React et approfondi son fonctionnement à travers une expérimentation concrète.

Enfin, nous avons analysé notre retour d'expérience afin de déterminer si cette technologie pouvait réellement être recommandée pour l'équipe et pour un produit destiné à évoluer dans le temps.

Le projet Kanban développé pendant cette période constitue donc principalement un support d'expérimentation. Il nous permet de confronter les informations obtenues pendant la veille à une utilisation réelle de la technologie.

1. Contexte et critères de décision

En tant qu'équipe technique, notre objectif n'est pas seulement de trouver une technologie permettant de réaliser une interface.

Nous devons également choisir une solution que l'équipe sera capable de comprendre, maintenir et faire évoluer.

Plusieurs critères ont donc été retenus :

- la courbe d'apprentissage ;
- les compétences déjà présentes dans l'équipe ;
- l'architecture proposée ;
- la gestion des composants et de l'état ;
- la qualité de la documentation ;
- la richesse de l'écosystème ;
- la facilité de maintenance ;
- la compatibilité avec TypeScript ;
- la communication avec une API ;

- la pérennité de la technologie ;
- le coût des dépendances ;
- la facilité d'intégration de nouveaux développeurs.

Les compétences de l'équipe constituent un critère important dans notre décision.

Une partie de l'équipe possède déjà des connaissances en React et se sent à l'aise avec son approche par composants.

Cela permet de réduire le temps de formation nécessaire et de concentrer davantage d'efforts sur le produit.

Cependant, nous ne souhaitons pas choisir React uniquement parce que nous le connaissons déjà. Notre décision doit également être défendue par des arguments techniques et organisationnels.

2. Comparaison initiale des technologies

Avant de prendre une décision, nous avons étudié React, Vue.js et Angular.

Cette première comparaison permet de comprendre la philosophie de chaque solution avant d'approfondir celle que nous avons réellement expérimentée.

2.1. React



FIGURE 1 – React

React est une bibliothèque JavaScript destinée à la création d'interfaces utilisateur. Son principe central repose sur la construction de l'interface à partir de composants.

Cette approche permet de découper une application en éléments plus petits et réutilisables. Chaque composant peut gérer une partie précise de l'interface et recevoir des informations provenant d'autres composants.

L'un des principaux avantages de React est sa flexibilité. Il peut être intégré dans différents types de projets et possède un écosystème très important. Son utilisation avec TypeScript est également bien adaptée aux applications modernes.

Cette flexibilité représente cependant aussi une limite. React ne définit pas seul toute l'architecture d'une application. L'équipe doit faire des choix concernant le routage, la récupération des données, l'organisation des fichiers ou encore certaines bibliothèques complémentaires.

Une mauvaise organisation peut donc conduire à des composants trop volumineux ou à une forte dépendance entre l'interface et les appels réseau.

Dans notre contexte, React présente toutefois un avantage humain important : plusieurs membres de l'équipe possèdent déjà des bases sur cette technologie. La montée en compétence nécessaire est donc plus faible.

2.2. Vue.js



FIGURE 2 – Vue.js

Vue.js est un framework JavaScript progressif destiné à la création d'interfaces web. Comme React, il repose fortement sur une logique de composants.

Vue propose une approche qui cherche à rester accessible tout en permettant de construire des applications complètes.

L'un de ses avantages est sa prise en main progressive. Sa syntaxe permet de regrouper de manière claire la structure, la logique et le style d'un composant.

Son système de réactivité permet également de mettre automatiquement à jour l'interface lorsque certaines données évoluent.

Vue possède son propre écosystème avec des solutions adaptées au routage, à la gestion d'état et au développement d'applications plus importantes.

Cette technologie aurait donc pu répondre aux besoins de notre produit.

Nous ne l'avons cependant pas expérimentée aussi profondément que React pendant cette veille. Nos conclusions concernant Vue reposent donc principalement sur la documentation et les recherches réalisées, et non sur une expérience équivalente de développement.

Dans notre équipe, Vue demanderait également un temps d'apprentissage supplémentaire, car les connaissances existantes sont davantage orientées vers React.

Ce point ne rend pas Vue moins intéressante techniquement, mais il influence le coût de son adoption dans notre contexte.

2.3. Angular



FIGURE 3 – Angular

Angular est un framework frontend complet permettant de développer des applications web structurées.

Contrairement à React, qui laisse davantage de liberté concernant l'organisation globale du projet, Angular propose un cadre plus important et de nombreuses conventions.

Cette approche peut représenter un avantage pour des applications complexes ou des équipes importantes. Les développeurs disposent d'une architecture plus encadrée et d'outils intégrés permettant d'organiser le projet.

Angular est également fortement associé à TypeScript, ce qui permet de bénéficier d'un typage statique dans l'ensemble de l'application.

Cette richesse entraîne cependant une courbe d'apprentissage plus importante. Un développeur doit comprendre plusieurs concepts et conventions propres au framework avant d'être pleinement productif.

Dans notre contexte, l'équipe possède moins d'expérience avec Angular. Son adoption demanderait donc davantage de temps de formation.

Nous n'avons pas développé notre expérimentation principale avec Angular. Comme pour Vue, notre analyse repose donc essentiellement sur notre veille documentaire.

Angular reste néanmoins une solution crédible, notamment dans un contexte nécessitant une architecture très encadrée et homogène.

3. Synthèse comparative

Le tableau suivant résume notre première analyse.

Critère	React	Vue.js	Angular
Approche par composants	Oui	Oui	Oui
TypeScript	Très bonne compatibilité	Compatible	Très intégré
Organisation	Flexible	Progressive	Très encadrée
Courbe d'apprentissage pour notre équipe	Plus faible	Intermédiaire	Plus importante
Liberté architecturale	Élevée	Élevée	Plus encadrée
Expérience de notre équipe	Bonne base	Limitée	Limitée
Expérimentation pendant la veille	Oui	Non	Non

TABLE 1 – Comparaison de React, Vue.js et Angular dans notre contexte

Ce tableau ne cherche pas à établir un classement universel.

Une technologie adaptée à notre équipe peut être moins pertinente dans une autre organisation.

Notre décision dépend donc du contexte du produit et de l'équipe.

4. Notre choix : React

Après cette première analyse, nous avons retenu **React** comme technologie principale pour notre frontend.

La première raison concerne les compétences de l'équipe. Plusieurs membres possèdent déjà des bases en React et comprennent son approche par composants. Nous pouvons donc réduire le temps nécessaire à la formation et atteindre plus rapidement un niveau de productivité acceptable.

Ce choix possède également une justification technique. Le modèle par composants correspond bien à notre volonté de séparer les différentes responsabilités de l'interface. React s'intègre également correctement avec TypeScript et dispose d'un écosystème suffisamment riche pour faire évoluer le produit.

Nous avons cependant identifié une limite importante : React laisse une grande liberté architecturale. Une équipe peut donc rapidement obtenir un projet difficile à maintenir si aucune convention n'est définie.

Notre choix de React implique donc une responsabilité supplémentaire : définir une architecture claire, limiter les dépendances inutiles et documenter les conventions utilisées.

Nous ne considérons pas React comme la meilleure solution dans tous les contextes. Nous considérons qu'elle présente, dans notre situation, un compromis cohérent entre compétences disponibles, rapidité de développement, architecture et capacité d'évolution.

5. React en profondeur

Après le choix initial, nous avons étudié React plus en profondeur à travers notre expérimentation.

5.1. La philosophie des composants

Le composant constitue l'un des éléments fondamentaux de React.

Au lieu de construire toute l'interface dans un seul fichier, nous pouvons la décomposer en plusieurs éléments.

Dans notre expérimentation, nous avons par exemple créé :

```
components/  
  KanbanColumn.tsx  
  LoginForm.tsx  
  RegisterForm.tsx  
  TaskCard.tsx
```

Chaque composant possède un rôle identifiable.

`LoginForm` est consacré au formulaire de connexion.

`RegisterForm` gère l'interface d'inscription.

`KanbanColumn` représente une colonne de l'interface expérimentale.

`TaskCard` représente une tâche.

Cette séparation améliore la compréhension du projet et permet d'éviter de concentrer toute l'interface dans un seul composant.

5.2. État et interactions

Une interface doit évoluer lorsque l'utilisateur effectue une action.

React utilise notamment l'état pour mémoriser les informations qui peuvent évoluer dans un composant.

Dans notre projet, nous utilisons par exemple :

```
const [email, setEmail] = useState('');  
const [password, setPassword] = useState('');  
const [message, setMessage] = useState('');
```

Lorsque l'utilisateur écrit dans un champ, l'état correspondant est mis à jour.

Cette approche permet à l'interface d'être directement liée aux données du composant.

Nous utilisons également l'état pour les listes, les tâches ou les messages affichés à l'utilisateur.

5.3. Flux de données

Dans notre architecture, les données circulent entre plusieurs niveaux.

Un composant parent peut transmettre des informations à un composant enfant grâce aux propriétés.

Par exemple, une colonne transmet une tâche au composant `TaskCard`.

Lorsqu'une tâche est modifiée, le composant peut ensuite prévenir son parent grâce à une fonction reçue en propriété.

Cette circulation permet de conserver une logique compréhensible, mais elle peut devenir plus complexe lorsque l'application grandit.

Pour notre expérimentation, une gestion d'état locale reste suffisante. Pour une application beaucoup plus importante, il serait nécessaire de réévaluer cette stratégie.

6. Pourquoi Next.js autour de React ?



FIGURE 4 – Next.js

Après avoir choisi React, nous avons utilisé Next.js pour structurer notre frontend.

Next.js est un framework basé sur React. Il ne remplace donc pas React dans notre comparaison avec Vue et Angular : il complète notre choix.

Son principal intérêt dans notre projet concerne l'organisation.

Nous utilisons l'App Router et une structure de dossiers permettant de représenter les différentes pages :

```
app/  
  board/  
    page.tsx  
  
  login/  
    page.tsx  
  
  register/  
    page.tsx  
  
  components/  
  services/
```

Cette structure rend les différentes responsabilités plus visibles.

Next.js permet également d'utiliser les composants React tout en apportant des fonctionnalités supplémentaires autour du routage et du rendu.

Ce choix possède néanmoins un coût de maintenance. Next.js évolue régulièrement et les pratiques recommandées peuvent changer.

Pour un produit maintenu plusieurs années, l'équipe devra donc suivre les évolutions du framework et éviter de migrer vers chaque nouvelle version sans analyser les bénéfices, les risques de régression et le coût de migration.

7. Axios : une dépendance volontaire

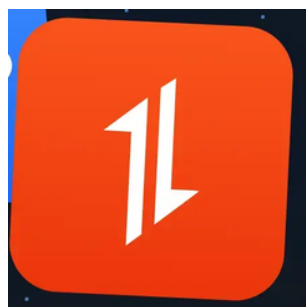


FIGURE 5 – Axios

Notre frontend doit communiquer avec une API backend.

Nous avons retenu Axios pour centraliser cette communication.

Axios n'était pas indispensable : l'API native `fetch` aurait également permis d'effectuer des requêtes HTTP.

Ajouter Axios signifie donc ajouter une dépendance supplémentaire au projet. Ce choix doit être justifié.

Dans notre cas, nous avons principalement recherché une configuration simple et commune pour les requêtes.

Nous avons créé :

```
services/  
  api.service.ts  
  auth.service.ts  
  cards.service.ts  
  lists.service.ts
```

Le fichier `api.service.ts` centralise la configuration Axios.

Les autres fichiers regroupent les requêtes selon leur responsabilité.

Cette architecture présente un autre avantage : nos composants ne dépendent pas directement de nombreux appels HTTP dispersés dans l'application.

Si notre manière de communiquer avec les données devait évoluer, les modifications seraient davantage concentrées dans la couche des services.

8. Architecture expérimentée

La veille technologique doit être confrontée à une utilisation réelle.

Nous avons donc expérimenté notre choix avec une application simple.

La figure suivante présente l'organisation réelle d'une partie du frontend.

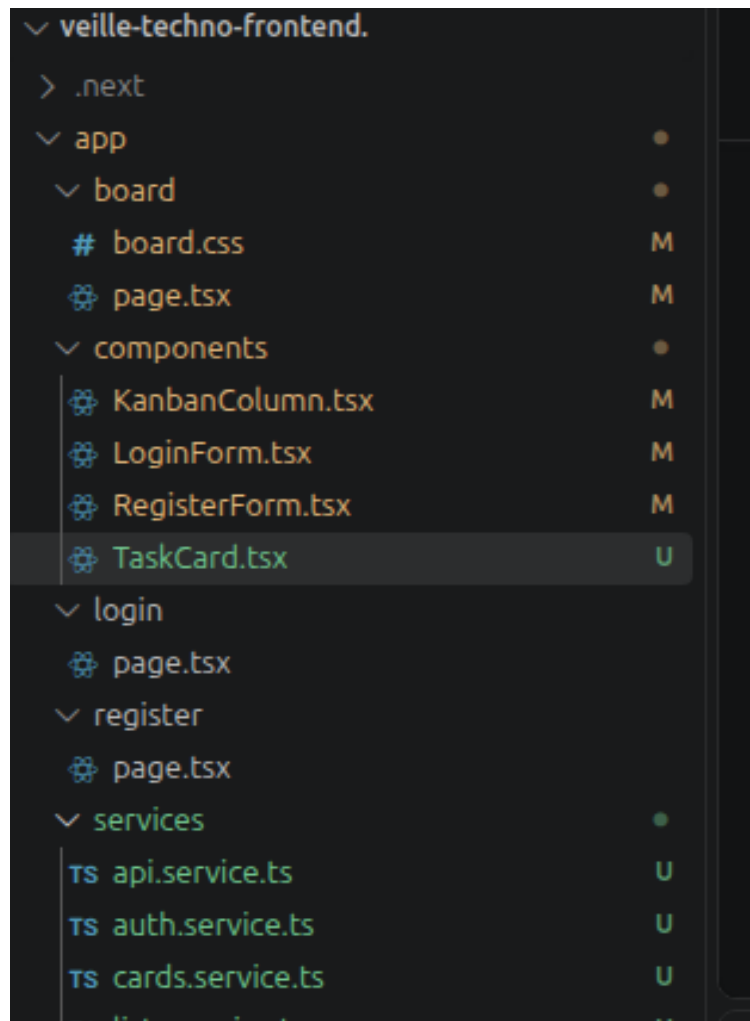


FIGURE 6 – Architecture réelle du frontend développée avec React et Next.js

Notre architecture peut être résumée de la manière suivante :

Pages → Composants → Services → API Backend

Les pages représentent les routes principales.

Les composants sont responsables de l'interface et des interactions.

Les services sont responsables de la communication avec les données.

Cette séparation constitue l'un des enseignements importants de notre expérimentation.

Au début, il est tentant de placer l'affichage, la gestion des formulaires et les appels API directement dans un même composant.

Cependant, cette solution devient rapidement difficile à comprendre.

La séparation entre composants et services permet de réduire cette confusion.

9. Retour d'expérience

Le projet expérimental nous a permis d'observer React et Next.js dans une situation réelle.

Le Kanban n'est pas l'objectif principal de cette veille. Il sert principalement à tester notre choix technologique.

9.1. Courbe d'apprentissage

La connaissance préalable de React a facilité le démarrage du projet.

Les notions de composants, d'état et d'événements étaient déjà partiellement connues.

La difficulté ne se trouvait donc pas uniquement dans la syntaxe de React, mais davantage dans la manière d'organiser correctement une application complète.

L'utilisation de Next.js a demandé de comprendre plus précisément l'App Router et l'organisation du dossier `app`.

Notre retour d'expérience montre donc qu'un développeur connaissant React pourra comprendre assez rapidement les composants du projet, mais devra également comprendre les conventions propres à Next.js.

9.2. Séparation des responsabilités

L'un des changements importants pendant le développement a concerné la séparation entre l'interface et les appels API.

Placer toute la logique dans les composants rendait le code plus difficile à comprendre.

Nous avons donc progressivement créé des services spécialisés.

Cette décision rejoint le principe de responsabilité unique : un module doit avoir une responsabilité principale identifiable.

Cette organisation facilite également la collaboration, car un développeur peut plus rapidement identifier l'endroit où se trouve la logique qu'il souhaite modifier.

9.3. Communication avec le backend

Notre expérimentation utilise une API backend existante.

Cette situation nous a permis de tester concrètement la communication entre le frontend et une source de données distante.

Nous avons notamment dû gérer les requêtes HTTP, l'authentification et la mise à jour de l'interface après une modification des données.

Cela nous a permis de constater l'importance de ne pas mélanger inutilement la logique réseau avec la logique d'affichage.

10. Organisation et priorisation des User Stories

Pour organiser le développement, nous avons découpé les fonctionnalités en *User Stories* puis défini leur priorité avec la méthode **MoSCoW**.

Cette méthode distingue les fonctionnalités indispensables (**Must Have**), importantes mais non bloquantes (**Should Have**) et celles pouvant être réalisées si le temps le permet (**Could Have**).

	A	B	C	D	E	F
	ID	User Story	Critères d'acceptation	Priorité MoSCoW	Justification de la priorité	Statut
1	FRONT-01	En tant que développeur, je veux initialiser le projet <code>React/Next.js</code> afin de disposer d'une base de développement fonctionnelle.	Le projet démarre correctement avec <code>Next.js</code> et <code>TypeScript</code> .	Must have	Sans initialisation du projet, aucune autre fonctionnalité <code>frontend</code> ne peut être développée.	Done
2						
3	FRONT-02	En tant que développeur, je veux organiser l'architecture du <code>frontend</code> afin de séparer clairement les pages, composants et services.	Les dossiers <code>app</code> , <code>components</code> et services sont organisés et les responsabilités sont séparées.	Must have	Indispensable pour garder un code compréhensible, faciliter le travail en équipe et séparer les responsabilités.	Done
4	FRONT-03	En tant qu'utilisateur, je veux disposer d'une interface d'inscription et de connexion afin d'accéder à l'application.	Les formulaires d'inscription et de connexion sont accessibles et valident les champs obligatoires.	Should have	L'authentification est utile avec notre API, mais le sujet <code>frontend</code> précise qu'elle n'est pas obligatoire pour le Kanban minimal.	Done
5	FRONT-04	En tant qu'utilisateur, je veux que l'authentification soit connectée à l'API afin d'utiliser mon compte.	L'inscription et la connexion appellent l'API et le <code>JWT</code> retourné est conservé côté <code>frontend</code> .	Should have	Nécessaire dans notre architecture avec API protégée, mais pas indispensable aux fonctionnalités minimales du sujet <code>frontend</code> .	Done
6	FRONT-05	En tant qu'utilisateur, je veux voir un tableau Kanban afin de visualiser mes colonnes et mes tâches.	Une page <code>Board</code> affiche une structure Kanban lisible avec des colonnes.	Must have	Le Kanban est le terrain d'expérimentation et la base des fonctionnalités demandées.	Done
7	FRONT-06	En tant qu'utilisateur, je veux afficher les colonnes provenant du <code>backend</code> afin de retrouver mes données persistantes.	Les colonnes sont récupérées depuis l'API et affichées dans le tableau.	Must have	Les colonnes sont nécessaires pour organiser les tâches et doivent être synchronisées avec notre <code>backend</code> .	Done
8	FRONT-07	En tant qu'utilisateur, je veux ajouter une colonne afin d'organiser mon tableau selon mon <code>workflow</code> .	Un formulaire permet de créer une colonne et celle-ci apparaît dans le tableau.	Must have	L'ajout de colonnes fait explicitement partie des fonctionnalités obligatoires du sujet.	In Progress
9	FRONT-08	En tant qu'utilisateur, je veux ajouter une tâche dans une colonne afin de suivre un travail à réaliser.	Une tâche avec titre et description peut être créée dans une colonne et enregistrée via l'API.	Must have	L'ajout de tâches aux colonnes est une fonctionnalité obligatoire du Kanban demandé.	Done

FIGURE 7 – Extrait de notre backlog de User Stories priorisé avec la méthode MoSCoW

Par exemple, la User Story **FRONT-07**, qui concerne l'ajout d'une colonne dans le Kanban, a été classée **Must Have**. Cette fonctionnalité fait partie du fonctionnement essentiel attendu dans notre expérimentation.

À l'inverse, certaines fonctionnalités apportant une amélioration sans bloquer le fonctionnement principal peuvent être classées **Should Have** ou **Could Have**.

Cette priorisation nous a permis de concentrer d'abord le développement sur les besoins essentiels et de mieux gérer le temps disponible.

11. Organisation du projet dans le temps

Le travail a été découpé en tickets dans GitHub Projects.

Title	Assignees	Status	Linked pull requests	Sub-issues progress	+
1 FRONT-01 - Initialize React project #1		Done			
2 FRONT-02 - Set up frontend architecture #2		Done			
3 FRONT-03 - Create authentication interface #3		Done			
4 FRONT-04 - Connect authentication to backend API #4		Done			
5 FRONT-05 - Create Kanban board interface #5		Done			
6 FRONT-06 - Display columns from backend #6		Done			
7 FRONT-08 - Add tasks to a column #8		Done			
8 FRONT-07 - Add a new column #7		In Progress			
9 FRONT-09 - Edit task title and description #9		In Progress			
10 FRONT-10 - Delete a task #10		Todo			
11 FRONT-11 - Improve responsive design #11		Todo			
12 FRONT-12 - Add advanced Kanban features #12		Todo			
13 FRONT-13 - Write project README #13		Todo			
14 FRONT-14 - Write frontend technology watch report #14		Todo			

FIGURE 8 – Organisation du développement dans GitHub Projects

Nous avons créé des tickets pour les principales fonctionnalités, puis des sous-tâches plus précises.

Cette organisation permet de mieux suivre l'avancement et de limiter les tickets contenant plusieurs responsabilités différentes.

Le découpage du travail a également évolué pendant le développement.

Certaines fonctionnalités qui semblaient simples au départ ont nécessité plusieurs étapes : création de l'interface, communication avec l'API puis vérification du comportement.

Cette expérience montre qu'une technologie possède également un coût en temps d'apprentissage et d'intégration.

Pour une équipe, cette information est aussi importante que les caractéristiques techniques décrites dans une documentation.

12. Maintenance, écosystème et pérennité

Choisir React et Next.js signifie également accepter de suivre leur évolution.

Un produit de start-up peut rester en développement pendant plusieurs années. Le choix technologique doit donc être évalué au-delà de la première version.

React possède un écosystème important, ce qui facilite l'accès à de nombreuses bibliothèques et ressources.

Cet avantage peut cependant devenir un risque si l'équipe ajoute trop de dépendances.

Chaque bibliothèque supplémentaire possède son propre cycle de maintenance, ses mises à jour et ses éventuelles incompatibilités.

Notre stratégie consiste donc à conserver un environnement aussi simple que possible et à justifier les dépendances réellement ajoutées.

Next.js nécessite également une surveillance particulière. Le framework évolue et introduit régulièrement de nouvelles possibilités.

Pour un produit en production, notre objectif ne serait pas de migrer immédiatement à chaque nouveauté, mais d'évaluer le bénéfice réel de la migration, son coût et son risque.

13. Collaboration et arrivée d'un nouveau développeur

Une technologie adaptée à une équipe doit permettre à un nouveau développeur de comprendre rapidement le projet.

Dans notre architecture, les responsabilités sont visibles dans l'arborescence.

Un développeur souhaitant travailler sur l'interface peut consulter le dossier **components**.

Un développeur souhaitant comprendre les communications avec le backend peut consulter **services**.

Les différentes pages sont regroupées dans le dossier **app**.

Cette organisation doit être accompagnée d'un README expliquant l'installation, les dépendances, les commandes et l'architecture générale.

L'objectif est d'éviter qu'une seule personne soit capable de comprendre ou de maintenir le projet.

14. Limites de notre étude

Notre veille possède également des limites qu'il est important d'identifier.

Nous avons réellement développé avec React et Next.js, mais nous n'avons pas réalisé le même projet complet avec Vue et Angular.

Nos observations sur React proviennent donc à la fois de la documentation et de notre expérimentation.

Nos observations sur Vue et Angular reposent principalement sur notre veille documentaire.

Nous n'avons pas non plus réalisé de benchmark complet permettant de conclure qu'une technologie est globalement plus performante qu'une autre.

Il serait donc incorrect de présenter React comme plus rapide que Vue ou Angular uniquement sur la base de notre expérimentation.

Cette distinction entre ce que nous avons lu et ce que nous avons réellement observé est importante pour conserver une veille crédible.

15. Recommandation finale

À l'issue de cette veille et de notre expérimentation, nous recommandons **React**, accompagné de **Next.js**, pour le frontend du produit dans le contexte actuel de notre équipe.

Cette décision repose d'abord sur les compétences disponibles. Plusieurs membres de l'équipe possèdent déjà des connaissances en React. Le coût de formation est donc inférieur à celui d'une adoption immédiate d'une technologie moins connue par l'équipe.

React apporte également une architecture par composants adaptée à notre manière d'organiser l'interface.

Next.js complète ce choix en apportant davantage de structure au projet.

Notre recommandation n'est cependant pas définitive.

Nous pourrions réévaluer cette décision si les besoins du produit évoluaient fortement, si l'équipe changeait ou si les coûts de maintenance de notre stack devenaient trop importants.

Angular pourrait par exemple devenir intéressant dans un contexte où l'équipe rechercherait un cadre beaucoup plus strict.

Vue pourrait également constituer une alternative pertinente pour une équipe possédant davantage d'expérience dans son écosystème.

Notre prescription est donc contextualisée :

React + Next.js + TypeScript

avec Axios comme dépendance HTTP actuellement retenue.

Ce choix permet aujourd'hui de combiner les compétences de l'équipe, une architecture par composants, un écosystème riche et une mise en œuvre suffisamment rapide pour notre contexte.

Conclusion

Cette veille technologique nous a montré qu'un choix frontend ne doit pas être réalisé uniquement à partir de la popularité d'une technologie.

React, Vue et Angular possèdent chacun une philosophie, des avantages et des contraintes différentes.

Notre responsabilité en tant qu'équipe technique consiste donc à mettre ces caractéristiques en relation avec la réalité du produit et de l'équipe.

Dans notre situation, React constitue un choix cohérent car l'équipe possède déjà des compétences dans cette technologie et parce que son approche par composants correspond à l'architecture recherchée.

Next.js nous permet ensuite d'apporter davantage de structure à cette base React.

L'expérimentation nous a également appris qu'un framework ne résout pas automatiquement les problèmes d'architecture.

La séparation des responsabilités, la gestion des dépendances, la documentation et l'organisation du projet restent des décisions de l'équipe.

Le Kanban nous a servi de terrain pour observer ces éléments dans une situation concrète.

Notre recommandation finale est donc d'utiliser React et Next.js pour le produit dans le contexte actuel, tout en continuant la veille afin de suivre leur évolution et de réévaluer ce choix lorsque les besoins du produit ou de l'équipe changeront.

Sources

Les informations documentaires utilisées dans cette veille proviennent en priorité des documentations officielles.

- React – Documentation officielle :
<https://react.dev/>
- Vue.js – Documentation officielle :
<https://vuejs.org/>
- Angular – Documentation officielle :
<https://angular.dev/>
- Next.js – Documentation officielle :
<https://nextjs.org/docs>
- TypeScript – Documentation officielle :
<https://www.typescriptlang.org/docs/>
- Axios – Documentation officielle :
<https://axios-http.com/docs/intro>